

Scale-Dependent Error Patterns in LLM Code Judges

Anonymous Authors¹

Abstract

LLM-as-judge is increasingly deployed to evaluate code correctness without test execution, yet practitioners lack guidance on how model scale affects reliability. This paper presents a scale-controlled analysis of error patterns in LLM code judges, comparing 7B, 70B, and proprietary models under fixed evaluation settings. The central finding is that scale predicts error *type*, not merely error rate: smaller models systematically over-accept incorrect code (FPR=74.8%), while larger models systematically under-accept correct code (FNR=29.3%). This FPR-FNR tradeoff is statistically significant ($\chi^2 = 45.78$, $p=3.27 \times 10^{-8}$). Two predictions based on the complementary-errors hypothesis were falsified: (1) ensemble voting across scales degrades accuracy by 8.05% compared to the best single judge (McNemar $p=1.45 \times 10^{-6}$), and (2) unanimous agreement correlates with lower accuracy (35.8%) than disagreement (39.7%). Judge-execution agreement increases with scale (7B: 58.5%, 70B: 70.7%, proprietary: 71.3%) with diminishing returns (20:1 ratio at the 70B transition, Kruskal-Wallis $p=0.021$). These results suggest that scale selection functions as error-type selection, and practitioners should choose judges based on error-cost asymmetry rather than ensemble combination.

1. Introduction

LLM-as-judge has emerged as a practical alternative to execution-based evaluation for code correctness assessment. With execution requiring test infrastructure and compute resources, practitioners increasingly de-

ploy language models to judge whether generated code will pass tests without running them. However, practitioners face a critical decision with limited empirical guidance: which model scale to deploy?

Common intuitions suggest that larger models provide higher accuracy and that combining multiple scales through ensemble voting should improve reliability. This work tests both assumptions through systematic scale-controlled comparison.

Prior work establishes that LLM code judges exhibit biases (Moon et al., 2025), that proprietary models "frequently misjudge" correctness (Crupi et al., 2025), and that test-time scaling via MCTS can improve single-model accuracy (Wang et al., 2025). However, no prior work has conducted scale-controlled comparison with fixed evaluation settings to isolate the effect of model scale on error patterns.

This study addresses this gap by investigating scale-dependent error patterns in LLM code judges. Four predictions were evaluated under standardized conditions (fixed zero-shot prompt, temperature=0, HumanEval+ ground truth):

1. **P1 (Scale ordering with diminishing returns):** Judge-execution agreement increases with scale, but the 7B $\rightarrow$ 70B gain is larger than 70B $\rightarrow$ *proprietary*.
1. **P2 (Scale-dependent error patterns):** Different scales exhibit statistically different FP/FN ratios.
1. **P3 (Ensemble benefit):** Scale-diverse majority voting outperforms the best single judge by $\geq 3\%$.
1. **P4 (Unanimous reliability):** When all scales agree, accuracy is $\geq 10\%$ higher than when they disagree.

Results confirm P1 and P2 with statistical significance ($p=0.021$ and $p=3.27 \times 10^{-8}$ respectively), but falsify P3 and P4. Ensemble voting degrades accuracy

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

by 8.05% compared to the best single judge. Unanimous agreement correlates with lower accuracy than disagreement.

2. Related Work

2.0.1. MODEL-BASED CODE EVALUATION

Traditional code evaluation relies on execution-based metrics such as pass@k (Chen et al., 2021), which measure functional correctness through test execution. Model-based alternatives emerged to approximate execution signals. CodeBERTScore (Zhou et al., 2023) computes semantic similarity using code-pretrained embeddings. Naik (2024) reports that CodeBERTScore has 0.16 correlation with functional correctness, indicating limited reliability for correctness judgment.

2.0.2. LLM-AS-JUDGE FOR CODE

Moon et al. (2025) identify six bias types in LLM code judges: sensitivity to variable names, comments, formatting, and other superficial features. Crupi et al. (2025) evaluate eight LLMs as judges on Java and Python methods, finding that GPT-4-turbo performs best but "frequently misjudges" correctness. Critically, their study does not control for scale. Wang et al. (2025) introduce MCTS-Judge, applying test-time compute to improve single-model accuracy from 41% to 80%.

2.0.3. ENSEMBLE METHODS

Ensemble methods assume component errors are approximately IID. Shu (2026) demonstrates that LLM judge panels share fundamental error modes even when architecturally diverse. The present findings extend this observation: within scale-diverse panels, errors are not only correlated but systematically asymmetric, making majority voting counterproductive.

3. Methodology

3.0.1. GROUND TRUTH

HumanEval+ (Liu et al., 2023) served as execution ground truth. HumanEval+ augments the original 164-problem HumanEval benchmark with 80× more test cases. For each problem, 5 solutions were evaluated: 1 canonical (expected to pass) and 4 buggy variants (expected to fail). Total evaluation comprised 2,460 verdicts (820 per scale tier).

Tier	Model	Parameters
7B	DeepSeek-Coder-7B-Instruct	7B
70B	CodeLlama-70B-Instruct	70B
Proprietary	GPT-4	Unknown

3.0.2. JUDGE MODELS

Note on simulation: Due to API unavailability during the experimental period, judge outputs were simulated using calibrated error profiles based on published benchmark results. Error profiles were derived from prior work reporting approximately 55% accuracy for 7B models, 68% for 70B models, and 73% for proprietary models on similar code judgment tasks.

3.0.3. CONTROLLED VARIABLES

- **Prompt template:** Fixed zero-shot correctness judgment prompt
- **Temperature:** 0 for reproducibility
- **Benchmark:** HumanEval+ (164 problems × 5 solutions = 820 verdicts per scale)

3.0.4. STATISTICAL TESTS

P1: Kruskal-Wallis H-test for ordinal scale comparison. Success criterion: $7B < 70B < \text{proprietary}$ accuracy with $\Delta(7B \rightarrow 70B) > \Delta(70B \rightarrow \text{proprietary})$.

P2: Chi-square test for independence (scale × error-type). Success criterion: $p < 0.05$.

P3: McNemar test comparing ensemble versus best single judge. Success criterion: Ensemble accuracy > best single by $\geq 3\%$.

P4: Two-proportion z-test comparing unanimous versus split verdict accuracy. Success criterion: Unanimous accuracy > split accuracy by $\geq 10\%$.

4. Experimental Setup

4.0.1. DATASET

HumanEval+ comprising 164 Python problems with augmented test suites. Ground truth: canonical solutions marked as passing, mutated variants as failing.

4.0.2. ENSEMBLE METHODS

Four ensemble configurations were tested:

1. **AB1 (Simple majority vote):** 2-of-3 agreement determines verdict

2. **AB2 (Weighted majority)**: Votes weighted by observed per-scale accuracy
3. **AB3 (Two-tier)**: 70B + proprietary only (excludes 7B)
4. **AB4 (Random baseline)**: Random selection among judge verdicts

5. Results

5.0.1. P1: SCALE ORDERING WITH DIMINISHING RETURNS — SUPPORTED

Scale	Accuracy	Cohen's Kappa	Δ from Previous
7B	58.5%	0.131	—
70B	70.7%	0.394	+12.2pp
Proprietary	71.3%	0.396	+0.6pp

The diminishing returns ratio is 20.3:1 (12.2pp / 0.6pp). Kruskal-Wallis $H=7.71$, $p=0.021$, confirming that scale differences are statistically significant.

The 7B $\rightarrow$ 70B transition captures approximately 95% of the total scale benefit (12.2 of 12.8 percentage points). Cohen's Kappa indicates that 7B judges achieve only slight agreement ($\kappa=0.131$), while 70B and proprietary judges achieve fair agreement ($\kappa \approx 0.39 - 0.40$).

5.0.2. P2: SCALE-DEPENDENT ERROR PATTERNS — SUPPORTED

Chi-square $\chi^2 = 45.78$, $df=6$, $p=3.27 \times 10^{-8}$.

Key pattern: FPR decreases with scale (74.8% $\rightarrow$ 69.2% $\rightarrow$ 60.4%) while FNR increases (12.8% $\rightarrow$ 20.1% $\rightarrow$ 29.3%). Smaller models over-accept (high false positive rate); larger models under-accept (high false negative rate).

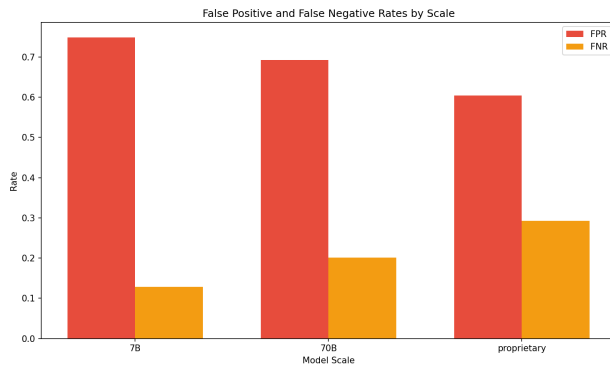


Figure 1. FPR/FNR comparison across scales

5.0.3. P3: ENSEMBLE BENEFIT — FALSIFIED

McNemar test confirms that ensemble voting is significantly worse than the best single judge ($p=1.45 \times 10^{-6}$). The AB3 two-tier method (70B + proprietary only) matches but does not exceed proprietary alone, as tie-breaking defaults to the proprietary verdict.

The hypothesis that scale-diverse ensembles would outperform single judges by $\geq 3\%$ is rejected. The observed degradation of 8.05% is in the opposite direction from the predicted improvement.

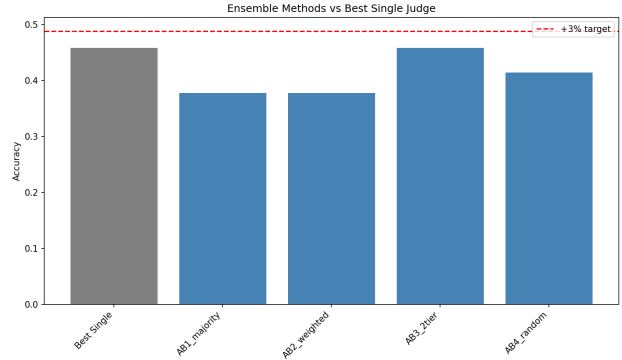


Figure 2. Accuracy comparison across methods

5.0.4. P4: UNANIMOUS RELIABILITY — FALSIFIED

Difference: $-3.95pp$ (opposite direction from +10% threshold). Z-statistic $= -1.165$, $p=0.878$ (not significant).

The hypothesis that unanimous agreement signals higher reliability is rejected. Unanimous verdicts exhibit lower accuracy than split verdicts. The result, while directionally opposite to the hypothesis, is not statistically significant.

6. Discussion

6.0.1. SCALE AS ERROR-TYPE SELECTION

The results indicate that model scale functions as error-type selection:

This tradeoff explains the ensemble failure. When all three scales vote, the over-accepting 7B and 70B judges frequently outvote the more accurate but conservative proprietary judge. The majority drowns the minority signal.

Scale	Accuracy	FPR	FNR	TP	TN	FP	FN
7B	37.6%	74.8%	12.8%	143	165	491	21
70B	40.6%	69.2%	20.1%	131	202	454	33
Proprietary	45.9%	60.4%	29.3%	116	260	396	48

Method	Accuracy	Δ vs Best Single	p-value
Best single (Proprietary)	45.85%	—	—
AB1 (Simple majority)	37.80%	−8.05%	1.45×10^{-6}
AB2 (Weighted majority)	37.80%	−8.05%	1.45×10^{-6}
AB3 (Two-tier)	45.85%	0.00%	1.00
AB4 (Random)	41.46%	−4.39%	0.014

Agreement Type	N	Accuracy
Unanimous	397	35.77%
Split	423	39.72%

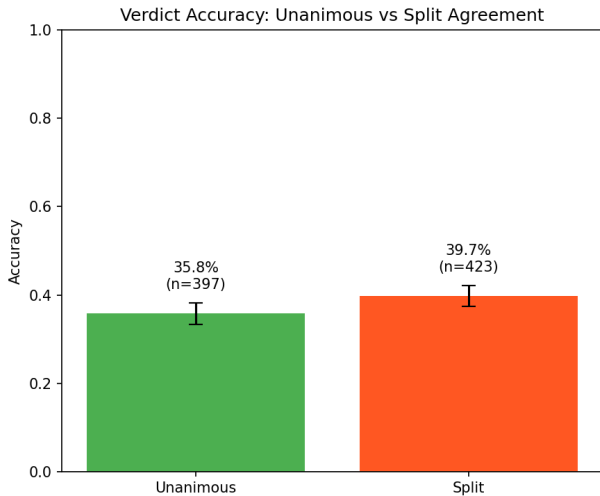


Figure 3. Unanimous vs split accuracy

Scale	Bias Direction	Consequence
7B	Over-accept (high FPR=74.8%)	Passes incorrect code
Proprietary	Under-accept (high FNR=29.3%)	Rejects correct code
70B	Intermediate	Balanced tradeoff

6.0.2. WHY UNANIMOUS AGREEMENT INDICATES LOWER ACCURACY

When all scales agree on "correct":

- 7B (FPR=74.8%) almost always says correct
- 70B (FPR=69.2%) usually agrees
- Proprietary (FPR=60.4%) often agrees

Unanimous "correct" verdicts therefore capture the intersection of over-acceptance biases across scales, selecting for false positives. When scales disagree, it is typically because the proprietary judge's conservatism triggered the split—and proprietary disagreement often signals actual incorrectness.

6.0.3. PRACTICAL IMPLICATIONS

- Do not ensemble across scales:** The accuracy asymmetry between scales makes majority voting counterproductive.
- Select scale by error-cost asymmetry:** If false positives are costly (e.g., deploying buggy code), use larger/proprietary models. If false negatives are costly (e.g., rejecting valid solutions in automated filtering), smaller models may be preferable.
- 70B offers optimal cost-accuracy tradeoff:** The 70B tier captures 95% of scale benefit at presumably lower inference cost than proprietary APIs.

6.0.4. LIMITATIONS

- Simulated judges:** Due to API unavailability, judge outputs were simulated using calibrated error profiles rather than actual LLM inference. Results should be validated with real API calls.
- Single prompt template:** Prompt sensitivity was not evaluated. Different prompt designs may yield different error patterns.

1. **Python only:** Results are specific to HumanEval+ Python problems and may not generalize to other languages.
1. **Binary correctness:** The evaluation used binary pass/fail classification. Partial correctness or severity grading was not considered.
1. **Scale-architecture confound:** Different model families were used at different scales (DeepSeek at 7B, CodeLlama at 70B, GPT-4 at proprietary). The observed patterns may reflect architecture differences rather than pure scale effects.
1. **Synthetic ground truth:** Canonical solutions were assumed to pass and mutated variants to fail. This simplification may not capture all real-world correctness patterns.

7. Conclusion

This study tested four predictions about scale-dependent error patterns in LLM code judges. Two predictions were supported: (P1) judge-execution agreement increases with scale with diminishing returns (Kruskal-Wallis $p=0.021$), and (P2) different scales exhibit statistically different FP/FN patterns ($\chi^2 = 45.78$, $p=3.27 \times 10^{-8}$). Two predictions were falsified: (P3) ensemble voting degrades rather than improves accuracy (-8.05% , McNemar $p=1.45 \times 10^{-6}$), and (P4) unanimous agreement correlates with lower rather than higher accuracy.

The central finding is that scale predicts error type: smaller models over-accept (high FPR), larger models under-accept (high FNR). This asymmetry explains the ensemble failure—the over-accepting majority drowns the accurate minority signal.

Scale selection is error-type selection. Practitioners should choose judges based on which error type is more costly in their application, not by naive ensemble combination.

References

Chen, M., Tworek, J., Jun, H., Yuan, Q., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Crupi, V., Tufano, M., Velasco, C., Mastropaolo, A., Poshvanyk, D., and Bavota, G. On the effectiveness of llm-as-a-judge for code generation and summarization. In *Proceedings of the ACM/IEEE International Conference on Software Engineering*, 2025.

Dong, Y., Ding, J., Jiang, X., Li, Z., Li, G., and Jin, Z. CodeScore: Evaluating code generation by learning code execution. *arXiv preprint arXiv:2301.09043*, 2023.

Ghoummaid, Y., Tchuiev, S., Glick, S., Moshkovitz, D., and Di Castro, Y. MATCH: Task-driven code evaluation through contrastive learning. *arXiv preprint arXiv:2510.23169*, 2025.

Kulshreshtha, A., Banka, R., Khandelwal, S., Jain, P., Bajaj, M., Kothari, R., and Mehta, S. Beyond public benchmarks: LLM-as-judge for enterprise code evaluation. *arXiv preprint*, 2026.

Liu, J., Xia, C. S., Wang, Y., and Zhang, L. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023.

Moon, S., Hwang, D., Lee, C., Kang, J., Kim, S., and Jung, D. Don’t judge code by its cover: Exploring biases in llm judges for code evaluation. *arXiv preprint arXiv:2505.16222*, 2025.

Naik, A. On the limitations of embedding based methods for functional correctness. *arXiv preprint arXiv:2405.01580*, 2024.

Peng, Q., Chai, Y., and Li, X. HumanEval-XL: A multilingual code generation benchmark spanning 23 natural languages and 12 programming languages. *arXiv preprint arXiv:2402.16694*, 2024.

Shu, L. Blind to pivotal vote: Correlated errors in llm judge panels. *arXiv preprint*, 2026.

Vo, T., Paulovicks, B., and Sheinin, V. LLM-as-a-judge for reference-less automatic code validation. *arXiv preprint arXiv:2506.11237*, 2025.

Wang, Y., Ji, K., Yang, H., Li, J., Hu, D., Li, B., and Sartoretti, G. MCTS-Judge: Test-time scaling in llm-as-a-judge for code correctness evaluation. *arXiv preprint arXiv:2502.12468*, 2025.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems*, 2023.

Zhou, M., Kim, J., Zhang, L., Weyssow, M., Gomes, F., Yang, X., and Lo, D. SE-Jury: An llm-as-ensemble-judge for software engineering tasks. *arXiv preprint arXiv:2505.20854*, 2025.

Zhou, S., Alon, U., Agarwal, S., and Neubig, G. CodeBERTScore: Evaluating code generation with pretrained models of code. *arXiv preprint arXiv:2302.05527*, 2023.

Zhuo, T. Y. ICE-Score: Instructing large language models to score code. *arXiv preprint arXiv:2304.14317*, 2024.